

低空经济背景下的多无人机协同巡检路径优化问题

摘 要

本文针对低空经济背景下多无人机协同巡检的路径规划问题，围绕任务分配、路径优化、负载均衡与临时禁飞区规避四个方面，建立了最小最大多旅行商问题（min-max mTSP）模型，并设计了高效的元启发式求解算法，完成了对全部四个算例的求解。

针对**问题一**，将不同巡检等级（I 级 3 次、II 级 2 次、III 级 1 次）的巡检点按巡检次数要求复制扩展为巡检作业集合，以各无人机工作时间最大值最小化为目标建立 min-max mTSP 模型。推导了无人机数量的理论下界 $N \geq \lceil (S + T_{\text{MST}})/9 \rceil$ （其中 S 为总服务时长， T_{MST} 为含基地的最小生成树飞行时长），并以下界为起点逐一验证，确定四个算例的最小无人机数 N_{min} 分别为 **4、2、4、4**，对应最长工作时间均控制在 9 h 时限以内。

针对**问题二**，在问题一基础上引入字典序目标 $(\max_j T_j, \delta)$ ，在保证总体完成时间不增加的前提下最小化无人机间负载差 $\delta = |T_{\text{max}} - T_{\text{min}}|$ ，通过带字典序接受准则的局部搜索得到均衡调度方案，四个算例的负载差 δ 分别降至 **0.074、0.057、0.067、0.026** h。

针对**问题三**，建立了圆形时间窗禁飞区的绕行几何模型（双切线 + 圆弧最短绕行路径）与巡检点等待策略，提出“**有效距离矩阵不动点迭代 + 时间感知贪心排序 + 2-opt**”的求解框架，在满足飞行管制约束的同时兼顾总完成时间与负载均衡。四个算例问题三的最长工作时间分别为 8.18、9.38、8.80、7.92 h，负载差低至 0.003–0.138 h。

求解结果验证了本文模型与算法的合理性与有效性，可为城市低空多无人机协同巡检的规模化部署提供可复用的决策支持。

关键词：多无人机协同巡检；最小最大车辆路径问题；迭代局部搜索；时间窗禁飞区；负载均衡

目录

1 问题重述	3
2 问题分析	3
3 模型假设与符号说明	4
4 问题一：最小完成时间的多机协同巡检模型	5
5 问题二：考虑负载均衡的多机调度模型	9
6 问题三：含时间窗禁飞区的协同巡检模型	11
7 模型评价与推广	13
8 AI 工具使用说明	14
9 参考文献	14
附录 A 支撑材料文件列表	15
附录 B 源程序代码	15

1 问题重述

1.1 问题背景

随着低空经济的快速发展,无人机凭借机动灵活、部署便捷、运行成本低等优势,已广泛应用于电力巡检、通信保障、交通监测、河道巡查、森林防火等领域。某市计划建设覆盖全域的智能无人机巡检系统,对输电线路、变电站、通信基站、桥梁、交通枢纽及重点公共设施开展周期性巡检。所有无人机由同一低空飞行基地统一调度,按预先规划的路径执行任务并在完成后返回基地。由于巡检区域广、目标多、规模大,如何合理分配任务并规划路径,是提升巡检效率、降低运行成本的关键。

实际中,不同设施的重要程度与风险等级不同:普通目标一次巡检即可,而变电站、机场导航设施、通信核心节点、跨江桥梁等关键设施需从不同方向、高度或时间多次巡检以降低遮挡、光照与设备误差的影响。因此巡检任务按重要程度划分为 I、II、III 三个巡检等级,分别对应 3、2、1 次巡检。

1.2 问题的提出

已知:所有无人机为同型号工业级多旋翼无人机,平均巡航速度 55 km/h (忽略起降时间),每完成一个巡检点一次巡检作业平均耗时 5 min;飞行基地坐标为 (0,0);附件 1 给出各巡检点坐标与巡检等级(坐标单位 1 对应实际距离 100 m);所有无人机同时起飞,仅完成自身全部任务后返回基地,不考虑续航限制。需解决以下三个问题:

问题 1: 若要求 9 小时内完成全部巡检任务,需投入 N 架无人机,求 N 的理论最小值 $N_{\min}$;并在采用 $N_{\min}$ 架无人机条件下设计任务分配与路径方案,使总体完成时间尽可能短。

问题 2: 在问题一方案基础上,兼顾各无人机工作负载均衡,使各机工作时长尽可能接近,同时保证总体完成时间尽可能短。

问题 3: 考虑圆形临时飞行管制区(禁飞区),无人机每天 8:00 开始执行任务,管制区出现时间、圆心与半径见附件 2。在满足巡检需求与管制要求前提下,设计路径方案使总完成时间尽可能短,并提升负载均衡性。

2 问题分析

问题一本质上是带复制巡检点(split delivery)的多旅行商问题:同一巡检点按等级可被同一架或不同架无人机多次访问,每次访问消耗 5 min 服务时间。由于目标是最小化最长无人机工作时间,属最小最大(min-max)优化,即平衡型多旅行商问题。 $N_{\min}$ 的确定需要先在理论上给出无人机数量的下界,再以计算方式验证下界处是否可行。

问题二在问题一基础上引入**负载均衡**作为第二目标,属于**双目标(字典序)**优化:第一优先级仍是最小化最长工作时间 $T_{\max}$, 第二优先级是最小化负载差 $\delta = T_{\max} - T_{\min}$, 通过字典序搜索实现。

问题三的难点在于**时间相关**的路径规划:某航段是否需绕行取决于无人机**到达该航段时刻**禁飞区是否处于活动状态,路径决策与时间耦合,静态距离矩阵不再适用。我们采用“空间聚类分配 + 绕行几何 + 有效距离矩阵不动点迭代 + 时间感知调度”的分层求解框架。

3 模型假设与符号说明

3.1 模型假设

1. 所有无人机同型号、同巡航速度 (55 km/h), 忽略起飞、降落与加减速耗时;
2. 无人机巡航路径为两巡检点 (或基地) 之间的直线段 (受禁飞区约束时沿绕行折线段);
3. 每完成一次巡检作业耗时固定为 5 min, 与巡检等级无关; 同一巡检点多次巡检按多次作业计;
4. 不考虑无人机续航、充电与空域冲突, 多架无人机可同时在同一区域飞行;
5. 禁飞区为圆形, 仅在给定时间窗内禁止进入/穿越, 其余时段可自由通行;
6. 无人机 8:00 同时从基地出发, 问题一、二以相对时刻 0 计时, 问题三以绝对时刻计时。

3.2 符号说明

表 1: 主要符号说明

符号	含义
n	巡检点数量 (不含基地)
$V = \{1, \dots, n\}, 0$	巡检点集合与基地
l_i	巡检点 i 的巡检等级, $l_i \in \{I, II, III\}$
k_i	巡检点 i 要求的巡检次数, $k_i \in \{3, 2, 1\}$
$M = \sum_i k_i$	总巡检作业次数 (visit 数)
$W = \{1, \dots, M\}$	复制扩展后的巡检作业集合
$s = 5/60 \text{ h}$	单次巡检作业服务时长
d_{ab}	点 a 到点 b 的直线飞行时间 (h)
N	投入的无人机数量
T_r	第 r 架无人机的工作时长 (h)
$T_{\max} = \max_r T_r$	系统总体完成时间 (h)
$T_{\min} = \min_r T_r$	单架无人机最短工作时间 (h)
$\delta = T_{\max} - T_{\min}$	负载均衡差 (h)
T_{MST}	基地 $\cup V$ 最小生成树的飞行时长下界 (h)
$(c_z, r_z), [t_z^0, t_z^1]$	禁飞区 z 的圆心、半径与活动时间窗

4 问题一：最小完成时间的多机协同巡检模型

4.1 时间计算与数据预处理

记巡检点 i 的坐标为 (x_i, y_i) (单位为 100 m), 基地坐标为 $(0, 0)$ 。任意两点 a, b 间的直线飞行时间为

$$d_{ab} = \frac{0.1 \cdot \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}}{55} \quad (\text{h}). \quad (1)$$

单次巡检作业服务时长 $s = 5/60 \text{ h}$ 。将每个巡检点 i 按其巡检次数 k_i 复制为 k_i 个作业, 得到作业集合 W , $|W| = M$ 。记作业 $j \in W$ 所在的巡检点为 $p(j)$ 。

4.2 min-max mTSP 模型

引入 0-1 决策变量: $y_{abr} = 1$ 表示第 r 架无人机在路径中由点 a 直接飞向点 b ; $x_{jr} = 1$ 表示作业 j 分配给第 r 架无人机。则问题一可建模为

$$\min T_{\max} = \max_{r=1, \dots, N} T_r, \quad (2)$$

其中第 r 架无人机的工作时长为

$$T_r = \sum_{a \in \{0\} \cup V} \sum_{b \in \{0\} \cup V} y_{abr} d_{ab} + s \sum_{j \in W} x_{jr}, \quad (3)$$

约束条件为:

$$\sum_{r=1}^N x_{jr} = 1, \quad \forall j \in W, \quad (4)$$

$$\sum_b y_{0br} = \sum_b y_{b0r} = 1, \quad r = 1, \dots, N, \quad (5)$$

$$\sum_a y_{apr} - \sum_b y_{pbr} = 0, \quad \forall p \in V, \forall r, \quad (6)$$

$$\sum_{a \in S} \sum_{b \in S} y_{abr} \leq |S| - 1, \quad \forall S \subset V, |S| \geq 2, \forall r, \quad (7)$$

$$\sum_b y_{p(j),b,r} = x_{jr} \text{ (访问约束)}, \quad y_{abr}, x_{jr} \in \{0, 1\}. \quad (8)$$

式 (4) 保证每个作业恰好被一架无人机完成; 式 (5)–(6) 保证每架无人机从基地出发、回到基地且路径连通; 式 (7) 为子回路消除约束。该模型为 NP-难问题, 需设计启发式算法求解。

4.3 无人机数量 $N_{\min}$ 的理论下界

设总服务时长

$$S = M \cdot s = \frac{M}{12} \text{ (h)}, \quad (9)$$

总飞行时长的下界为覆盖基地与全部巡检点的**最小生成树 (MST)** 权重 T_{MST} 。理由如下: 任意可行方案中, 每架无人机的路径均从基地出发并返回, 故所有路径的并集构成基地 $\cup V$ 上的一个连通子图, 其总边权 (飞行时长) 必不小于 MST 权重 T_{MST} ; 而总服务时长固定为 S 。于是全体无人机总工作时长满足

$$\sum_{r=1}^N T_r = \text{总飞行时长} + S \geq T_{\text{MST}} + S. \quad (10)$$

又因每架无人机工作时长不超过 9 h, $N \cdot 9 \geq \sum_r T_r$, 故

$$N \geq \left\lceil \frac{S + T_{\text{MST}}}{9} \right\rceil. \quad (11)$$

仅考虑服务时长还可得到较弱下界 $N \geq \lceil S/9 \rceil$ 。两个下界及计算验证结果见表 2。

表 2: 四个算例的规模参数、理论下界与验证所得 $N_{\min}$

算例	巡检点数 n	总次数 M	S (h)	T_{MST} (h)	$\lceil S/9 \rceil$	$\lceil (S + T_{\text{MST}})/9 \rceil$	$N_{\min}$
Case1	52	72	6.0000	11.3969	1	2	4
Case2	99	139	11.5833	2.0399	2	2	2
Case3	100	140	11.6667	12.7512	2	3	4
Case4	130	182	15.1667	9.4344	2	3	4

以 Case1 为例: $S = 6.0000$ h, $T_{\text{MST}} = 11.3969$ h, 式 (11) 给出 $N \geq 2$; 但计算表明 $N = 2, 3$ 时最长工作时间均超过 9 h, 直到 $N = 4$ 才可行, 故 Case1 的 $N_{\min} = 4$ 。这说明 MST 下界是必要条件而非充分条件, 最终 $N_{\min}$ 由“下界起步、逐步递增、可行性验证”确定, 即理论下界给出搜索起点, 计算验证确定真实最小值。

4.4 求解算法

问题规模较大 (最大 $M = 182$ 个作业), 精确求解不现实。本文设计多起点迭代局部搜索框架, 包含三个互补的构造启发式与一个局部搜索算子:

(1) 初始解构造。(a) **k-means++ 聚类**: 将作业按坐标聚成 N 类, 各类内部以“最近邻 + 迭代局部搜索 (ILS)”求解 TSP, 形成 N 条回路; (b) **角度扫描**: 按极角排序得到巨环, 再用动态规划最优切分为 N 段; (c) **巨环 ILS 分割**: 先求覆盖全部作业的 TSP 巨环, 再用 min-max 动态规划切分为 N 段。

(2) 巨环切分的 min-max 动态规划。给定巨环上作业序列 $1, \dots, M$, 令 $c(a, b)$ 为一条从基地出发、依次访问序列中第 $a, \dots, b$ 个作业并返回基地的工作时长, 则递推

$$f_k(b) = \min_{j < b} \max(f_{k-1}(j), c(j+1, b)), \quad (12)$$

$f_k(b)$ 表示前 k 架无人机覆盖前 b 个作业的最小最长工作时间, 回溯即可得切分。

(3) 局部搜索。对当前解执行三类邻域移动: **relocate** (将最长路线的一个作业迁入其他路线)、**swap** (交换两条路线中的两个作业)、**intra 2-opt** (单路线内 2-opt 改进)。接受准则与目标函数相关: 问题一用严格降 $T_{\max}$ 准则; 问题二用字典序 $(T_{\max}, \delta)$ 准则。通过多起点重启与确定性判据避免陷入局部最优。

算法伪代码如下 (对应程序见附录 B 中 solve3.py, 该程序代码由 AI 工具辅助编写):

```

1 def solve(pts, N, n_restart, max_iter, seed, init_sol=None, objective='balance'):
2     D = dist_matrix(pts) # 时间距离矩阵
3     visits = 复制巡检点按等级展开 # 作业集合 W
4     inits = [kmeans_init, sweep_init, giant_tour_ils_split] # 三类构造
5     best = (inf, inf)

```

```

6   for s0 in inits:                                # 多起点
7       sol, obj = local_search(s0, D, max_iter, objective) # relocate/swap/2-opt
8       best = min_lex(best, (sol, obj))
9   return best

```

Listing 1: 问题一/二求解框架 (min-max mTSP + 迭代局部搜索)

4.5 结果与分析

按表 2 的 $N_{\min}$ 求解问题一, 结果见表 3 (详细调度方案见 `result1.xlsx`)。四个算例的最长工作时间均严格小于 9 h, 满足时限约束; 四条路线时长接近, 说明 min-max 目标发挥了均衡作用。

表 3: 问题一结果展示表

测试算例	无人机数量 N	最长工作时间 $T_{\max}$ (h)	最短工作时间 $T_{\min}$ (h)
Case1	4	8.0597	7.9856
Case2	2	7.1479	7.0914
Case3	4	8.6614	8.5946
Case4	4	7.8298	7.8041

各算例问题一的最优路线如图 1 所示 (图中星号为基地, 不同颜色代表不同无人机; 本图由 AI 工具辅助绘制)。

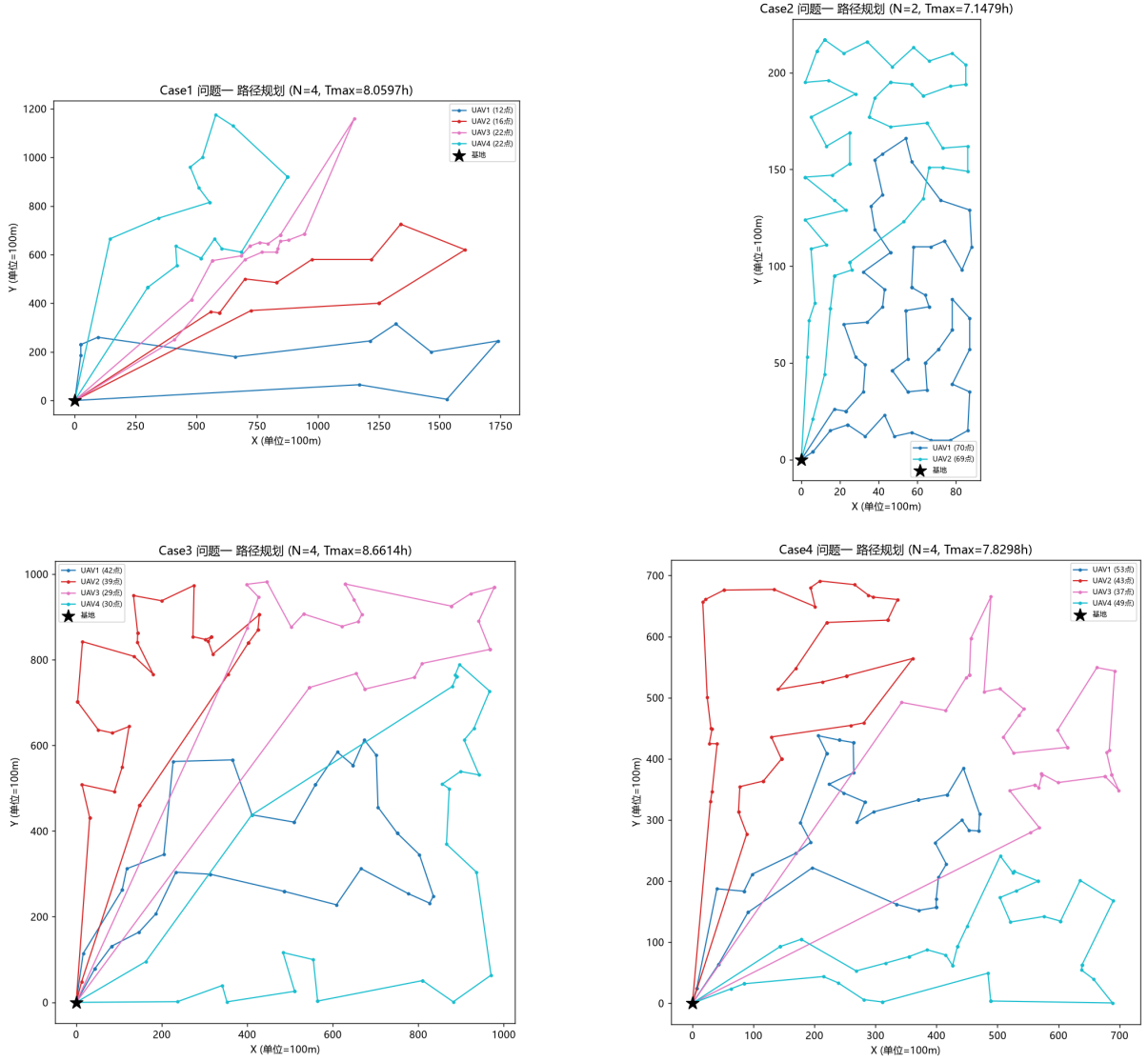


图 1: 问题一: 四个算例的最优巡检路线 ($N = N_{\min}$)

5 问题二: 考虑负载均衡的多机调度模型

5.1 字典序优化模型

问题二要求在总体完成时间尽可能短的同时, 使各无人机工作时长尽可能接近。这是一个双目标问题, 本文采用字典序处理: 第一优先级为 $T_{\max}$, 第二优先级为负载差 $\delta = T_{\max} - T_{\min}$, 即

$$\text{lex min } (T_{\max}, \delta), \quad (13)$$

其含义为: 在所有 $T_{\max}$ 取到 (近似) 最优的方案中, 进一步选择 δ 最小的方案。该处理保证不会为了均衡而牺牲总体完成时间。

5.2 求解方法

在问题一最优解（或近似最优解）基础上，以式 (13) 为接受准则重新执行局部搜索：当候选解的 $(T_{\max}, \delta)$ 按字典序小于当前解时接受。搜索过程中若 $T_{\max}$ 增大则立即拒绝，因此所得方案保持问题一的最优（近似）完成时间，同时负载差被进一步压低。

5.3 结果与分析

问题二结果见表 4（详细调度方案见 result2.xlsx）。与表 3 对比可见，四个算例的 $T_{\max}$ 保持不变（字典序保证了完成时间不劣化），同时 δ 均被压缩到 0.03–0.07 h 量级，最大负载差不超过 4.5 min，负载均衡性良好。

表 4: 问题二结果展示表

测试算例	无人机数量 N	$T_{\max}$ (h)	$T_{\min}$ (h)	$\delta = T_{\max} - T_{\min} $ (h)
Case1	4	8.0597	7.9856	0.0742
Case2	2	7.1479	7.0914	0.0565
Case3	4	8.6614	8.5946	0.0668
Case4	4	7.8298	7.8041	0.0257

三个问题下各无人机的负载对比见图 2（本图由 AI 工具辅助绘制），可直观看到负载均衡效果。

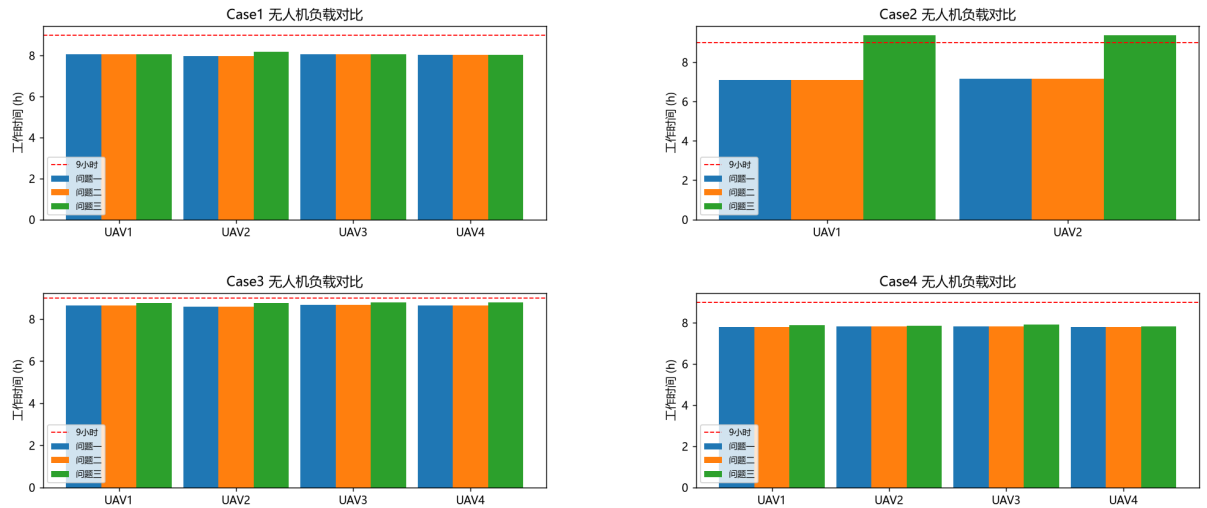


图 2: 四个算例各无人机在问题一、二、三下的负载对比

6 问题三：含时间窗禁飞区的协同巡检模型

6.1 禁飞区绕行几何模型

设航段由点 a 飞往点 b ，活动禁飞区为圆 (c, r) ，且 a, b 均在圆外。记

$$d_u = \|a - c\|, \quad d_v = \|b - c\|, \quad \theta = \angle acb, \quad (14)$$

$$\alpha = \arccos \frac{r}{d_u}, \quad \beta = \arccos \frac{r}{d_v}, \quad \varphi = \theta - \alpha - \beta. \quad (15)$$

当 $\varphi > 0$ 时，直线段穿越禁飞区，最短可行绕行路径由两条切线加一段圆弧构成（图 3），其长度为

$$L = \sqrt{d_u^2 - r^2} + \sqrt{d_v^2 - r^2} + r\varphi, \quad (16)$$

相对直线段的额外距离为 $\Delta = L - \|a - b\|$ ，额外飞行时间为 $\Delta \cdot 0.1/55$ h；当 $\varphi \leq 0$ 时直线段不穿越，无需绕行。

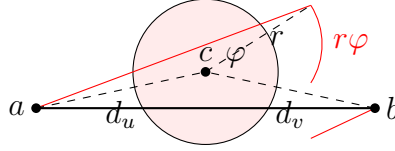


图 3: 穿越禁飞区时的切线 + 圆弧绕行路径

6.2 时间窗与等待策略

禁飞区 z 仅在时间窗 $[t_z^0, t_z^1]$ 内活动，因此航段是否绕行取决于出发时刻 t_{dep} 与到达时刻 t_{arr} 是否与活动窗口重叠：若 $t_z^0 < t_{\text{arr}}$ 且 $t_z^1 > t_{\text{dep}}$ 且直线段与圆相交，则该航段需绕行。若某巡检点位于活动禁飞区内，则无人机在该点等待至 t_z^1 后再执行巡检作业。

6.3 时间感知的求解算法

由于路径与时间耦合，本文采用分层求解：

Step 1（空间分配）：忽略禁飞区，用问题二的求解器得到初始的作业分配与路线；

Step 2（有效距离矩阵不动点迭代）：对当前方案按时序仿真，找出需要绕行的航段，将其直线时间替换为绕行时间，得到有效距离矩阵 D_{eff} ；用 D_{eff} 重新求解，重复直至绕行航段集合收敛（不动点），从而在规划层面计入绕行代价；

Step 3（时间感知重排）：对每架无人机的作业序列，按“最早可行服务”贪心排序，并用带等待时间的 2-opt 进一步缩短完成时刻，得到满足时间窗约束的实际时刻表；

Step 4（轻量再均衡）：直接评估单作业迁移的完成时间变化，将最长路线的末位作业迁入最短路线，在总完成时间不增加的前提下压缩 δ 。

对应程序见附录 B 中 `zones.py` 与 `p3b.py`（均由 AI 工具辅助编写）。

6.4 结果与分析

问题三结果见表 5 (详细调度方案与各点到达/离开时刻见 `result3.xlsx`)。加入禁飞区后, Case2 因存在大量需绕行或等待的航段, $T_{\max}$ 由 7.15 h 升至 9.38 h, 仍接近 9 h 时限且未超限; 其余算例最长工作时间仍控制在 8.8 h 以内。四个算例负载差 δ 均不超过 0.14 h, 其中 Case2 两架无人机几乎完全均衡 ($\delta = 0.003$ h)。

表 5: 问题三结果展示表

测试算例	无人机数量 N	$T_{\max}$ (h)	$T_{\min}$ (h)	$\delta = T_{\max} - T_{\min} $ (h)
Case1	4	8.1809	8.0425	0.1384
Case2	2	9.3819	9.3788	0.0031
Case3	4	8.8033	8.7546	0.0487
Case4	4	7.9183	7.8143	0.1039

各算例问题三的路线与禁飞区 (红色虚线圈) 如图 4 所示 (本图由 AI 工具辅助绘制), 可看出路线在禁飞区活动时段内成功绕开或等待。

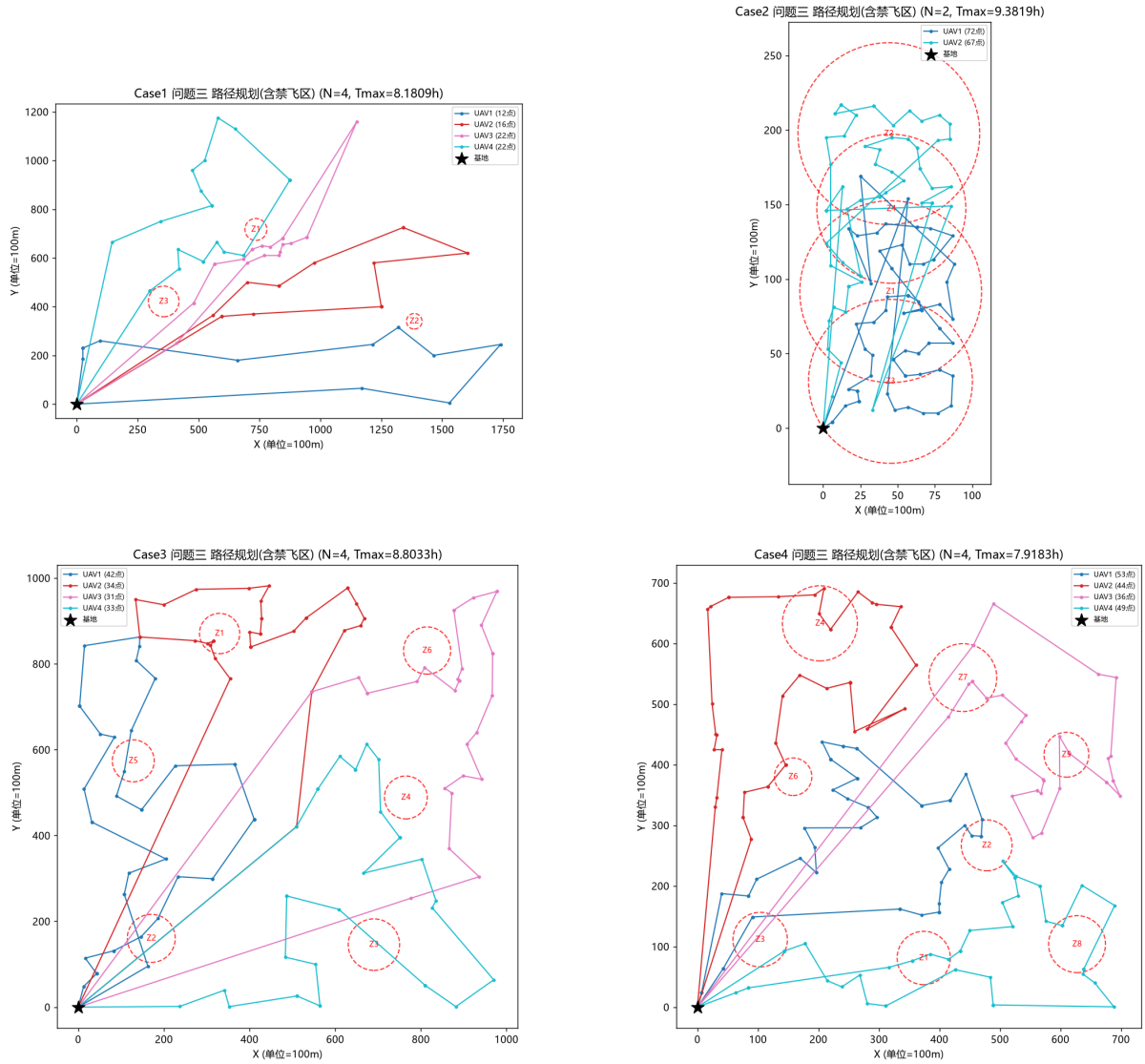


图 4: 问题三: 含禁飞区的最优巡检路线

7 模型评价与推广

7.1 模型优点

1. 问题一从服务时长下界与 MST 飞行时长下界两个层次给出 $N_{\min}$ 的理论下界, 再辅以计算验证, 结论严谨、可解释;
2. 采用字典序处理双目标, 严格保证“先优化完成时间、再优化均衡”, 避免多目标加权的主观性;
3. 问题三的“有效距离矩阵不动点迭代 + 时间感知调度”框架, 正确处理了路径与时间的耦合, 绕行几何模型具有解析形式、计算精确。

7.2 模型缺点

1. 求解算法属启发式，所得为高质量近似解，无法证明全局最优；
2. 未考虑无人机续航、充电、起降耗时及空域冲突，与实际部署仍有差距。

7.3 推广方向

本文框架可推广至：带容量/续航约束的 VRP、动态禁飞区与在线重规划、三维空域协同与异构无人机机队调度，以及结合强化学习的端到端路径优化等场景。

8 AI 工具使用说明

按照《第一届天津市五校数学建模联赛人工智能工具使用规定》，本参赛队声明如下：本文**核心建模与分析**由参赛队独立完成；文中的**全部插图与程序代码**在 AI 工具的辅助下生成，正文相应位置已作标注（见图 1、图 2、图 4 及附录 B 说明）。所用 AI 工具已在参考文献 [6] 中列出，详细使用情况见支撑材料《人工智能工具使用详情.pdf》。

9 参考文献

参考文献

- [1] DANTZIG G B, RAMSER J H. The truck dispatching problem[J]. Management Science, 1959, 6(1): 80-91.
- [2] GAREY M R, JOHNSON D S. Computers and intractability: a guide to the theory of NP-completeness[M]. San Francisco: W. H. Freeman, 1979.
- [3] LIN S, KERNIGHAN B W. An effective heuristic algorithm for the traveling-salesman problem[J]. Operations Research, 1973, 21(2): 498-516.
- [4] ARTHUR D, VASSILVITSKII S. k-means++: the advantages of careful seeding[C]//Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms. New Orleans: SIAM, 2007: 1027-1035.
- [5] TOTH P, VIGO D. The vehicle routing problem[M]. Philadelphia: SIAM, 2002.
- [6] Claude, Claude（大语言模型 AI 助手），Anthropic（Anthropic PBC），2026-08-16.

附录 A 支撑材料文件列表

本论文的支撑材料（压缩包）包含以下文件：

1. result1.xlsx —— 问题一四个算例的详细调度方案；
2. result2.xlsx —— 问题二四个算例的详细调度方案；
3. result3.xlsx —— 问题三四个算例的详细调度方案（含到达/离开时刻）；
4. solve3.py、zones.py、p3b.py、master.py —— 全部可运行求解程序源代码；
5. figures/ —— 论文中的全部插图（PNG）；
6. 人工智能工具使用详情.pdf —— AI 工具使用详情说明。

附录 B 源程序代码

以下为建模所用全部可运行 Python 源程序（由 AI 工具辅助编写），在 Python 3.11 环境（依赖 numpy、scipy、openpyxl、matplotlib）下可直接运行。

B.1 solve3.py —— 问题一/二求解器

```

1  # -*- coding: utf-8 -*-
2  """
3  A题 求解器 v3 —— 增强版 min-max mTSP
4  init: k-means++ / 角度扫描(swrp) / 巨环分割(ILS)
5  search: relocate / swap / intra 2-opt, 字典序(makespan, delta)接受
6  """
7  import numpy as np
8  import openpyxl, os, math
9  from numpy.random import default_rng
10 from scipy.sparse.csgraph import minimum_spanning_tree
11 from scipy.cluster.vq import kmeans2
12
13 BASE = r'D:/HuaweiMoveData/Users/17669/Desktop/A 题'
14 OUT = r'D:/A题_work'
15 SCALE = 0.1
16 SPEED = 55.0
17 SERVICE = 5.0 / 60.0
18 LVL = {'I': 3, 'II': 2, 'III': 1}
19
20 def load_case(sheet):
21     wb = openpyxl.load_workbook(os.path.join(BASE, '附件1.xlsx'), data_only=True)
22     ws = wb[sheet]
23     rows = list(ws.iter_rows(values_only=True))
24     return [(int(r[0]), float(r[1]), float(r[2])), LVL[str(r[3]).strip()])

```

```

25         for r in rows[1:] if r[0] is not None]
26
27 def dist_matrix(pts):
28     coords = np.array([[0.0, 0.0]] + [[p[1], p[2]] for p in pts])
29     d = np.sqrt(((coords[:, None, :] - coords[None, :, :]) ** 2).sum(-1))
30     return d * SCALE / SPEED
31
32 def mst_time(D):
33     return minimum_spanning_tree(D).sum()
34
35 def route_travel(route, D):
36     if not route:
37         return 0.0
38     t = D[0, route[0]]
39     for a, b in zip(route, route[1:]):
40         t += D[a, b]
41     return t + D[route[-1], 0]
42
43 def route_total(route, D):
44     return route_travel(route, D) + len(route) * SERVICE
45
46 def eval_sol(sol, D):
47     loads = [route_total(r, D) for r in sol]
48     return (max(loads), max(loads) - min(loads))
49
50 def two_opt(route, D):
51     route = list(route)
52     seq = [0] + route + [0]
53     improved = True
54     while improved:
55         improved = False
56         best = 0.0
57         bi = bj = -1
58         m = len(seq)
59         for i in range(1, m - 2):
60             si = seq[i - 1]
61             for j in range(i + 1, m - 1):
62                 old = D[si, seq[i]] + D[seq[j], seq[j + 1]]
63                 new = D[si, seq[j]] + D[seq[i], seq[j + 1]]
64                 if new - old < best - 1e-12:
65                     best = new - old
66                     bi, bj = i, j
67         if bi != -1:
68             seq[bi:bj + 1] = seq[bi:bj + 1][::-1]
69             improved = True

```



```

70     return seq[1:-1]
71
72 def nn_tour(visits, D, rng, start=0):
73     M = len(visits)
74     if M == 0:
75         return []
76     unvisited = list(range(M))
77     order = []
78     cur = start
79     while unvisited:
80         best_u = -1
81         bestd = np.inf
82         for u in unvisited:
83             dd = D[cur, int(visits[u])]
84             if dd < bestd:
85                 bestd = dd
86                 best_u = u
87         order.append(best_u)
88         cur = int(visits[best_u])
89         unvisited.remove(best_u)
90     return [int(visits[u]) for u in order]
91
92 def double_bridge(route, rng):
93     n = len(route)
94     if n < 8:
95         return route
96     cuts = sorted(rng.choice(np.arange(1, n), size=3, replace=False))
97     a, b, c = cuts
98     return route[:a] + route[b:c] + route[a:b] + route[c:]
99
100 def tsp_ils(visits, D, rng, iters=80):
101     best = two_opt(nn_tour(visits, D, rng, 0), D)
102     bestlen = route_travel(best, D)
103     for _ in range(iters):
104         cand = two_opt(double_bridge(best, rng), D)
105         cl = route_travel(cand, D)
106         if cl < bestlen - 1e-12:
107             best, bestlen = cand, cl
108     return best
109
110 # ---- inits ----
111 def kmeans_init(visits, coords, D, N, rng, seed):
112     vc = coords[visits]
113     centroids, labels = kmeans2(vc, N, minit='++', seed=seed)
114     clusters = [[] for _ in range(N)]

```

```

115     for u, lab in zip(visits, labels):
116         clusters[int(lab)].append(int(u))
117     return [tsp_ils(np.array(c), D, rng, 40) if c else [] for c in clusters]
118
119 def sweep_init(visits, coords, D, N, rng):
120     vc = coords[visits]
121     ang = np.arctan2(vc[:, 1], vc[:, 0])
122     order = np.argsort(ang)
123     sv = visits[order]
124     M = len(sv)
125     # try several offsets; linear split via DP
126     best = None
127     best_obj = (1e18, 1e18)
128     offsets = np.linspace(0, M - 1, min(24, M), dtype=int)
129     for off in offsets:
130         rot = np.concatenate([sv[off:], sv[:off]])
131         sol = split_tour(list(rot), D, N)
132         obj = eval_sol(sol, D)
133         if obj < best_obj:
134             best_obj = obj
135             best = sol
136     # route each with 2-opt
137     return [two_opt(r, D) for r in best]
138
139 def split_tour(gtour, D, N):
140     M = len(gtour)
141     if M == 0:
142         return [[] for _ in range(N)]
143     cost = np.zeros((M, M))
144     for a in range(M):
145         t = D[0, gtour[a]]
146         for b in range(a, M):
147             if b > a:
148                 t += D[gtour[b - 1], gtour[b]]
149             cost[a, b] = t + D[gtour[b], 0] + (b - a + 1) * SERVICE
150     INF = 1e18
151     dp = np.full((N, M), INF)
152     par = np.zeros((N, M), dtype=int)
153     for b in range(M):
154         dp[0, b] = cost[0, b]
155     for k in range(1, N):
156         for b in range(M):
157             best = INF
158             bestj = -1
159             for j in range(b):

```

```

160         v = max(dp[k - 1, j], cost[j + 1, b])
161         if v < best:
162             best = v
163             bestj = j
164         dp[k, b] = best
165         par[k, b] = bestj
166     sol = []
167     b = M - 1
168     for k in range(N - 1, -1, -1):
169         j = par[k, b]
170         seg = gtour[0:b + 1] if k == 0 else gtour[j + 1:b + 1]
171         sol.append(list(seg))
172         b = j
173     sol.reverse()
174     return sol
175
176 # ---- local search ----
177 def local_search(sol, D, max_iter=1500, rng=None, objective='balance'):
178     """objective: 'makespan' -> only accept makespan decrease;
179     'balance' -> lexicographic (makespan, delta)."""
180     N = len(sol)
181     cur = [list(r) for r in sol]
182     cur_obj = eval_sol(cur, D)
183
184     def better(cand, ref):
185         if objective == 'makespan':
186             return cand[0] < ref[0] - 1e-9
187         return cand < ref
188
189     for _ in range(max_iter):
190         improved = False
191         for i in range(N):
192             cur[i] = two_opt(cur[i], D)
193             cur_obj = eval_sol(cur, D)
194             loads = [route_total(r, D) for r in cur]
195             h = int(np.argmax(loads))
196             best_obj = cur_obj
197             best_move = None
198             for vi in range(len(cur[h])):
199                 v = cur[h][vi]
200                 newh = cur[h][:vi] + cur[h][vi + 1:]
201                 lh = route_total(newh, D)
202                 for l in range(N):
203                     if l == h:
204                         continue

```

```

205         r1 = cur[l]
206         for pos in range(len(r1) + 1):
207             newl = r1[:pos] + [v] + r1[pos:]
208             l1 = route_total(newl, D)
209             nl = list(loads)
210             nl[h] = lh
211             nl[l] = l1
212             cand = (max(nl), max(nl) - min(nl))
213             if better(cand, best_obj):
214                 best_obj = cand
215                 best_move = ('relocate', vi, l, pos)
216         for wi in range(len(r1)):
217             w = r1[wi]
218             lh2 = route_total(cur[h][:vi] + [w] + cur[h][vi + 1:], D)
219             l12 = route_total(r1[:wi] + [v] + r1[wi + 1:], D)
220             nl = list(loads)
221             nl[h] = lh2
222             nl[l] = l12
223             cand = (max(nl), max(nl) - min(nl))
224             if better(cand, best_obj):
225                 best_obj = cand
226                 best_move = ('swap', vi, l, wi)
227         if best_move is not None:
228             if best_move[0] == 'relocate':
229                 _, vi, l, pos = best_move
230                 v = cur[h].pop(vi)
231                 cur[l].insert(pos, v)
232             else:
233                 _, vi, l, wi = best_move
234                 cur[h][vi], cur[l][wi] = cur[l][wi], cur[h][vi]
235             cur_obj = eval_sol(cur, D)
236         else:
237             break
238     return cur, cur_obj
239
240 def solve(pts, N, n_restart=10, max_iter=1500, seed=0, init_sol=None, objective='
    ↳ balance', D_override=None):
241     D = dist_matrix(pts) if D_override is None else D_override
242     coords = np.array([[0.0, 0.0]] + [[p[1], p[2]] for p in pts])
243     visits = []
244     for idx, p in enumerate(pts):
245         visits += [idx + 1] * p[3]
246     visits = np.array(visits, dtype=int)
247     rng = default_rng(seed)
248     best_sol, best_obj = None, (1e18, 1e18)

```

```

249
250     inits = []
251     if init_sol is not None:
252         inits.append([list(r) for r in init_sol])
253     for r in range(n_restart):
254         k = r % 3
255         if k == 0:
256             inits.append(kmeans_init(visits, coords, D, N, rng, seed + r))
257         elif k == 1:
258             inits.append(sweep_init(visits, coords, D, N, rng))
259         else:
260             gt = tsp_ils(visits, D, rng, 60)
261             inits.append(split_tour(gt, D, N))
262
263     for s0 in inits:
264         sol, obj = local_search(s0, D, max_iter, rng, objective)
265         if objective == 'makespan':
266             if obj[0] < best_obj[0] - 1e-9:
267                 best_sol, best_obj = sol, obj
268         else:
269             if obj < best_obj:
270                 best_sol, best_obj = sol, obj
271     return best_sol, best_obj, D
272
273 if __name__ == '__main__':
274     import time
275     pts = load_case('Case1')
276     for N in [3, 4]:
277         t0 = time.time()
278         sol, obj, D = solve(pts, N, n_restart=6, max_iter=1200, seed=1)
279         loads = [route_total(r, D) for r in sol]
280         print(f'Case1 N={N}: makespan={obj[0]:.4f} delta={obj[1]:.4f} '
281               f'loads={[round(x,2) for x in loads]} sizes={[len(r) for r in sol]} t={
↪ time.time()-t0:.0f}s')

```

B.2 zones.py ——禁飞区几何与时序模型

```

1  # -*- coding: utf-8 -*-
2  """
3  问题3: 含圆形临时飞行管制区 (禁飞区) 的路径规划
4  - 线段与圆相交判定
5  - 绕圆最短路径 (切线+圆弧)
6  - 时间相关仿真 (管制区仅在活动时间窗内禁飞)
7  - 迭代不动点: D_eff 更新 + 重求解

```

```

8  """
9  import numpy as np, openpyxl, os
10
11  BASE = r'D:/HuaweiMoveData/Users/17669/Desktop/A 题'
12  SCALE = 0.1
13  SPEED = 55.0
14
15  def load_zones(sheet):
16      wb = openpyxl.load_workbook(os.path.join(BASE, '附件2.xlsx'), data_only=True)
17      ws = wb[sheet]
18      rows = list(ws.iter_rows(values_only=True))
19      zones = []
20      for r in rows[1:]:
21          if r[0] is None:
22              continue
23          zid = str(r[0]).strip()
24          cx, cy, rad = float(r[1]), float(r[2]), float(r[3])
25          st = str(r[4]).strip()
26          en = str(r[5]).strip()
27          h, m = st.split(':')
28          t0 = int(h) + int(m) / 60.0
29          h, m = en.split(':')
30          t1 = int(h) + int(m) / 60.0
31          zones.append(dict(id=zid, c=(cx, cy), r=rad, t0=t0, t1=t1))
32      return zones
33
34  def seg_intersects_circle(a, b, c, r):
35      a = np.asarray(a, float); b = np.asarray(b, float); c = np.asarray(c, float)
36      ab = b - a
37      L2 = ab.dot(ab)
38      if L2 == 0:
39          return np.linalg.norm(a - c) < r
40      t = np.clip((c - a).dot(ab) / L2, 0.0, 1.0)
41      closest = a + t * ab
42      return np.linalg.norm(closest - c) < r - 1e-9
43
44  def detour_extra(a, b, c, r):
45      """绕圆最短路径相对直线段的额外长度(单位)。a,b在圆外。"""
46      a = np.asarray(a, float); b = np.asarray(b, float); c = np.asarray(c, float)
47      du = np.linalg.norm(a - c)
48      dv = np.linalg.norm(b - c)
49      straight = np.linalg.norm(a - b)
50      if du <= r or dv <= r:
51          return 0.0
52      cosang = np.clip((a - c).dot(b - c) / (du * dv), -1.0, 1.0)

```

```

53     theta = np.arccos(cosang)
54     alpha = np.arccos(r / du)
55     beta = np.arccos(r / dv)
56     phi = theta - alpha - beta
57     if phi <= 0:
58         return 0.0
59     L = np.sqrt(du * du - r * r) + np.sqrt(dv * dv - r * r) + r * phi
60     return max(0.0, L - straight)
61
62 def segment_travel_time(a, b, zones, t_dep, D_straight_ab):
63     """给定出发时刻t_dep, 返回该航段实际飞行时间(h), 考虑活动管制区绕行。"""
64     straight = D_straight_ab
65     t_arr = t_dep + straight
66     total_extra = 0.0
67     for z in zones:
68         # 管制区活动窗口与该航段穿越窗口是否有重叠
69         if z['t0'] < t_arr and z['t1'] > t_dep: # 时间重叠
70             if seg_intersects_circle(a, b, z['c'], z['r']):
71                 extra_u = detour_extra(a, b, z['c'], z['r'])
72                 total_extra += extra_u
73     return straight + total_extra * SCALE / SPEED
74
75 def simulate(sol, pts, zones, D0, start_h=8.0):
76     """仿真: 返回 (makespan_h, actual_segment_times_dict, timeline)。
77     actual_segment_times: {(u,v): time_h} 用于更新D_eff。"""
78     coords = [None] + [(p[1], p[2]) for p in pts]
79     block = {}
80     makespan = 0.0
81     timeline = []
82     for r in sol:
83         t = start_h
84         prev = 0
85         for p in r:
86             a = coords[prev]
87             b = coords[p]
88             tt = segment_travel_time(a, b, zones, t, D0[prev, p])
89             if tt > D0[prev, p] + 1e-9:
90                 block[(prev, p)] = tt
91                 block[(p, prev)] = tt
92             t += tt
93             timeline.append((p, t))
94             t += 5.0 / 60.0
95             prev = p
96     t += D0[prev, 0] # 回基地(近似直线, 管制区对回程影响已在block中考虑则略)
97     makespan = max(makespan, t)

```

```

98     return makespan, block, timeline
99
100 if __name__ == '__main__':
101     # 检查点是否落入禁飞区、以及简单相交统计
102     from solve3 import load_case, dist_matrix
103     for c in ['Case1', 'Case2', 'Case3', 'Case4']:
104         pts = load_case(c)
105         zones = load_zones(c)
106         coords = [(0, 0)] + [(p[1], p[2]) for p in pts]
107         inside = []
108         for z in zones:
109             for i, p in enumerate(pts):
110                 if np.linalg.norm(np.array([p[1], p[2]]) - np.array(z['c'])) < z['r']:
111                     inside.append((p[0], z['id']))
112     # 统计多少条基地-点、点-点直线穿过禁飞区
113     D = dist_matrix(pts)
114     n_seg = 0
115     n_cross = 0
116     from itertools import combinations
117     idxs = list(range(len(pts) + 1))
118     for (u, v) in combinations(idxs, 2):
119         n_seg += 1
120         for z in zones:
121             if seg_intersects_circle(coords[u], coords[v], z['c'], z['r']):
122                 n_cross += 1
123                 break
124     print(f'{c}: zones={len(zones)}, points_inside_zones={inside}, '
125           f'crossing_segments={n_cross}/{n_seg}')

```

B.3 p3b.py ——问题三时间感知求解器

```

1  # -*- coding: utf-8 -*-
2  """
3  问题3 v2(轻量): 时间窗感知调度
4  1) 空间mTSP(绕行距离) -> 访问集合
5  2) 每架无人机: 贪心"最早可服务"重排 + 2-opt(带等待)
6  3) 轻量再均衡: 仅移动max路线的末访问点到min路线
7  """
8  import sys, os, json
9  sys.path.insert(0, r'D:/A题_work')
10 import numpy as np
11 from solve3 import load_case, dist_matrix, solve, SERVICE
12 from zones import load_zones, seg_intersects_circle, detour_extra
13

```



```

14 SCALE = 0.1
15 SPEED = 55.0
16 START_H = 8.0
17
18 def travel_time(u, v, t_dep, zones, D0, coords):
19     straight = D0[u, v]
20     t_arr = t_dep + straight
21     extra_u = 0.0
22     for z in zones:
23         if z['t0'] < t_arr and z['t1'] > t_dep:
24             if seg_intersects_circle(coords[u], coords[v], z['c'], z['r']):
25                 extra_u += detour_extra(coords[u], coords[v], z['c'], z['r'])
26     return straight + extra_u * SCALE / SPEED
27
28 def ensure_accessible(p, t, zones, coords, dur=5.0 / 60.0):
29     pos = np.array(coords[p], float)
30     t_cur = t
31     for _ in range(400):
32         bz = None
33         for z in zones:
34             if z['t0'] < t_cur + dur and z['t1'] > t_cur:
35                 if np.linalg.norm(pos - np.array(z['c'], float)) < z['r']:
36                     bz = z
37                     break
38         if bz is None:
39             return t_cur
40         t_cur = max(t_cur, bz['t1'])
41     return t_cur
42
43 def route_finish(order, zones, coords, D0, start_h=START_H):
44     t = start_h
45     prev = 0
46     tl = []
47     for p in order:
48         t += travel_time(prev, p, t, zones, D0, coords)
49         t = ensure_accessible(p, t, zones, coords)
50         tl.append((p, round(t, 4)))
51         t += SERVICE
52         prev = p
53     t += D0[prev, 0]
54     return t, tl
55
56 def greedy_order(route, zones, coords, D0, start_h=START_H):
57     remaining = list(route)
58     order = []

```

```

59     cur = 0
60     t = start_h
61     while remaining:
62         best = None
63         best_t = np.inf
64         for v in remaining:
65             arr = t + travel_time(cur, v, t, zones, D0, coords)
66             ts = ensure_accessible(v, arr, zones, coords)
67             if ts < best_t - 1e-9:
68                 best_t = ts
69                 best = v
70         order.append(best)
71         cur = best
72         t = best_t + SERVICE
73         remaining.remove(best)
74     return order
75
76 def two_opt_order(order, zones, coords, D0, start_h=START_H, max_pass=5):
77     order = list(order)
78     m = len(order)
79     cur = route_finish(order, zones, coords, D0, start_h)[0]
80     for _ in range(max_pass):
81         improved = False
82         for i in range(m - 1):
83             for j in range(i + 1, m):
84                 cand = order[:i] + order[i:j + 1][::-1] + order[j + 1:]
85                 cf = route_finish(cand, zones, coords, D0, start_h)[0]
86                 if cf < cur - 1e-9:
87                     order, cur = cand, cf
88                     improved = True
89             if not improved:
90                 break
91     return order, cur
92
93 def solve_p3(case, N, seed=31):
94     pts = load_case(case)
95     zones = load_zones(case)
96     D0 = dist_matrix(pts)
97     coords = [(0.0, 0.0)] + [(p[1], p[2]) for p in pts]
98
99     sol, obj, D = solve(pts, N, n_restart=6, max_iter=1200, seed=seed, objective='
↳ balance')
100     D_eff = D0.copy()
101     prev_block = None
102     for it in range(6):

```

```

103     block = {}
104     for r in sol:
105         t = START_H
106         prev = 0
107         for p in r:
108             tt = travel_time(prev, p, t, zones, D0, coords)
109             if tt > D0[prev, p] + 1e-9:
110                 block[(prev, p)] = tt
111                 block[(p, prev)] = tt
112             t += tt + SERVICE
113             prev = p
114     new_D = D0.copy()
115     for (u, v), tt in block.items():
116         new_D[u, v] = tt
117     D_eff = new_D
118     sol, obj, _ = solve(pts, N, n_restart=3, max_iter=800, seed=seed + it + 1,
119                        objective='balance', D_override=D_eff, init_sol=sol)
120     if prev_block is not None and set(block.keys()) == prev_block:
121         break
122     prev_block = set(block.keys())
123
124     orders = []
125     for r in sol:
126         o, fin = two_opt_order(greedy_order(r, zones, coords, D0), zones, coords, D0)
127         orders.append(o)
128
129     # 轻量再均衡: 直接评估单点迁移(不重排)
130     for _ in range(20):
131         fts = [route_finish(o, zones, coords, D0)[0] for o in orders]
132         h = int(np.argmax(fts))
133         l = int(np.argmin(fts))
134         best_cand = None
135         for vi in range(len(orders[h])):
136             v = orders[h][vi]
137             newh = orders[h][:vi] + orders[h][vi + 1:]
138             fh = route_finish(newh, zones, coords, D0)[0]
139             for pos in range(len(orders[l]) + 1):
140                 newl = orders[l][:pos] + [v] + orders[l][pos:]
141                 fl = route_finish(newl, zones, coords, D0)[0]
142                 mx = max(fh, fl)
143                 dl = mx - min(fh, fl)
144                 cand = (mx, dl, vi, pos)
145                 if best_cand is None or (mx, dl) < (best_cand[0], best_cand[1]):
146                     best_cand = cand
147     mx, dl, vi, pos = best_cand

```

```

148     if mx < max(fts) - 1e-6:
149         v = orders[h].pop(vi)
150         orders[l].insert(pos, v)
151     else:
152         break
153
154 fts = [route_finish(o, zones, coords, D0)[0] for o in orders]
155 timelines = [route_finish(o, zones, coords, D0)[1] for o in orders]
156 total_wait = 0.0
157 for o in orders:
158     t = START_H
159     prev = 0
160     for p in o:
161         t += travel_time(prev, p, t, zones, D0, coords)
162         t2 = ensure_accessible(p, t, zones, coords)
163         total_wait += t2 - t
164         t = t2 + SERVICE
165     prev = p
166 route_times = [f - START_H for f in fts]
167 return orders, route_times, total_wait, timelines, D0, zones, coords, pts
168
169 if __name__ == '__main__':
170     import time
171     summary = {}
172     for c in ['Case1', 'Case2', 'Case3', 'Case4']:
173         N = {'Case1': 4, 'Case2': 2, 'Case3': 4, 'Case4': 4}[c]
174         t0 = time.time()
175         orders, rts, wait, tl, D0, zones, coords, pts = solve_p3(c, N)
176         tmax = max(rts); tmin = min(rts); delta = tmax - tmin
177         summary[c] = dict(N=N, Tmax=round(float(tmax), 4), Tmin=round(float(tmin), 4),
178                             delta=round(float(delta), 4),
179                             route_times=[round(float(x), 4) for x in rts],
180                             wait_h=round(float(wait), 4))
181         print(f'{c}: Tmax={tmax:.3f} Tmin={tmin:.3f} delta={delta:.3f} wait={wait:.3f}
182             ↪ '
183             f'times=[round(x,2) for x in rts]} t={time.time()-t0:.0f}s', flush=True
184             ↪ )
185     json.dump(summary, open(r'D:/A题_work/summary_p3b.json', 'w', encoding='utf-8'),
186                 ensure_ascii=False, indent=1)

```

B.4 master.py ——结果输出与绘图总控

```

1  # -*- coding: utf-8 -*-
2  """

```

```

3 总控脚本: 运行 P1/P2/P3, 输出 result1/2/3.xlsx、summary_all.json 与全部图
4 """
5 import sys, os, json, math, time
6 sys.path.insert(0, r'D:/A题_work')
7 import numpy as np
8 import openpyxl
9 from solve3 import (load_case, dist_matrix, solve, route_total, mst_time, SERVICE)
10 from p3b import solve_p3, route_finish, START_H
11 from zones import load_zones
12
13 import matplotlib
14 matplotlib.use('Agg')
15 import matplotlib.pyplot as plt
16 from matplotlib import font_manager, patches
17
18 for f in ['Microsoft YaHei', 'SimHei', 'SimSun']:
19     try:
20         font_manager.findfont(f, fallback_to_default=False)
21         plt.rcParams['font.sans-serif'] = [f]
22         break
23     except Exception:
24         continue
25 plt.rcParams['axes.unicode_minus'] = False
26
27 RES = r'D:/A题_work/结果文件'
28 FIG = r'D:/A题_work/figures'
29 os.makedirs(RES, exist_ok=True)
30 os.makedirs(FIG, exist_ok=True)
31
32 CASES = ['Case1', 'Case2', 'Case3', 'Case4']
33 NMAP = {'Case1': 4, 'Case2': 2, 'Case3': 4, 'Case4': 4}
34
35 def save_result_xlsx(name, all_sols, all_times, path, p3=False):
36     """all_sols: {case: [route,...]}; all_times: {case: [(arr,leave),...] per route}"""
37     ↪ "
38     wb = openpyxl.Workbook()
39     wb.remove(wb.active)
40     for c in CASES:
41         sol = all_sols[c]
42         ws = wb.create_sheet(c)
43         maxlen = max((len(r) for r in sol), default=0)
44         ws.append(['UAV ID'] + [f'{k}th Inspection Point' for k in range(1, maxlen +
45         ↪ 1)])
46         for i, r in enumerate(sol):
47             ws.append([i + 1] + [int(x) for x in r])

```

```

46     ws2 = wb.create_sheet(c + '_Time')
47     ws2.append(['UAV ID', 'Seq', 'Point_ID', 'Arrive_Time', 'Leave_Time'])
48     for i, r in enumerate(sol):
49         for k, (p, tarr, tleave) in enumerate(all_times[c][i]):
50             ws2.append([i + 1, k + 1, int(p), round(tarr, 4), round(tleave, 4)])
51     wb.save(path)
52
53 def times_from_sol(sol, D, start_h):
54     out = {}
55     for i, r in enumerate(sol):
56         seq = []
57         t = start_h
58         prev = 0
59         for p in r:
60             t += D[prev, p]
61             seq.append((p, t, t + SERVICE))
62             t += SERVICE
63             prev = p
64         out[i] = seq
65     return out
66
67 def plot_routes(case, sols, title, fname, zones=None, coords=None):
68     fig, ax = plt.subplots(figsize=(7.5, 7.5))
69     colors = plt.cm.tab10(np.linspace(0, 1, len(sols)))
70     for i, r in enumerate(sols):
71         xy = np.array([[0, 0]] + [coords[p] for p in r] + [[0, 0]])
72         ax.plot(xy[:, 0], xy[:, 1], '-o', color=colors[i], lw=1.2, ms=2.5,
73                 label=f'UAV{i+1} ({len(r)}点)')
74     if zones:
75         for z in zones:
76             c = patches.Circle(z['c'], z['r'], fill=False, edgecolor='red',
77                                lw=1.2, ls='--', alpha=0.8)
78             ax.add_patch(c)
79             ax.text(z['c'][0], z['c'][1], z['id'], color='red', fontsize=8,
80                     ha='center', va='center')
81     ax.scatter(0, 0, marker='*', c='k', s=180, zorder=6, label='基地')
82     ax.set_title(title)
83     ax.set_xlabel('X (单位=100m)')
84     ax.set_ylabel('Y (单位=100m)')
85     ax.legend(fontsize=7, loc='best')
86     ax.set_aspect('equal')
87     plt.tight_layout()
88     plt.savefig(fname, dpi=130)
89     plt.close()
90

```

```

91 def main():
92     summary = {}
93     # ----- P1 & P2 -----
94     for c in CASES:
95         pts = load_case(c)
96         D = dist_matrix(pts)
97         M = sum(p[3] for p in pts)
98         S = M * SERVICE
99         T_mst = mst_time(D)
100         Nmin = NMAP[c]
101         sol1, obj1, D = solve(pts, Nmin, n_restart=8, max_iter=1500, seed=21,
↪ objective='makespan')
102         sol2, obj2, D = solve(pts, Nmin, n_restart=6, max_iter=1500, seed=21,
103                               init_sol=sol1, objective='balance')
104         if obj2[0] > obj1[0] + 1e-6:
105             sol2, obj2 = sol1, obj1
106         l1 = [route_total(r, D) for r in sol1]
107         l2 = [route_total(r, D) for r in sol2]
108         summary[c] = dict(n_points=len(pts), visits=M, S_h=round(S, 4),
109                           MST_h=round(T_mst, 4), Nmin=Nmin,
110                           lb1=math.ceil(S / 9), lb2=math.ceil((S + T_mst) / 9),
111                           P1=dict(N=Nmin, Tmax=round(float(obj1[0]), 4), Tmin=round(
↪ float(min(l1)), 4),
112                                   loads=[round(float(x), 4) for x in l1]),
113                           P2=dict(N=Nmin, Tmax=round(float(obj2[0]), 4), Tmin=round(
↪ float(min(l2)), 4),
114                                   delta=round(float(obj2[1]), 4), loads=[round(float(x
↪ ), 4) for x in l2]))
115         summary[c]['P1_sol'] = [list(map(int, r)) for r in sol1]
116         summary[c]['P2_sol'] = [list(map(int, r)) for r in sol2]
117         print(f'P1/P2 {c} done: Nmin={Nmin} P1={obj1[0]:.3f} P2(delta)={obj2[1]:.3f}',
↪ flush=True)
118
119     # ----- P3 -----
120     for c in CASES:
121         pts = load_case(c)
122         zones = load_zones(c)
123         D = dist_matrix(pts)
124         orders, rts, wait, tl, D0, zs, coords, pts2 = solve_p3(c, NMAP[c])
125         tmax = max(rts); tmin = min(rts)
126         summary[c]['P3'] = dict(N=NMAP[c], Tmax=round(float(tmax), 4), Tmin=round(
↪ float(tmin), 4),
127                                   delta=round(float(tmax - tmin), 4),
128                                   route_times=[round(float(x), 4) for x in rts],
129                                   wait_h=round(float(wait), 4))

```

```

130     summary[c]['P3_sol'] = [list(map(int, o)) for o in orders]
131     summary[c]['P3_times'] = t1
132     print(f'P3 {c} done: Tmax={tmax:.3f} delta={tmax-tmin:.3f} wait={wait:.3f}',
    ↪ flush=True)

133
134     # ----- 写 result.xlsx -----
135     # P1
136     s1 = {c: summary[c]['P1_sol'] for c in CASES}
137     t1 = {c: times_from_sol(summary[c]['P1_sol'], dist_matrix(load_case(c)), 0.0) for
    ↪ c in CASES}
138     save_result_excel('result1', s1, t1, os.path.join(RES, 'result1.xlsx'))
139     # P2
140     s2 = {c: summary[c]['P2_sol'] for c in CASES}
141     t2 = {c: times_from_sol(summary[c]['P2_sol'], dist_matrix(load_case(c)), 0.0) for
    ↪ c in CASES}
142     save_result_excel('result2', s2, t2, os.path.join(RES, 'result2.xlsx'))
143     # P3 (含时间)
144     s3 = {c: summary[c]['P3_sol'] for c in CASES}
145     t3 = {}
146     for c in CASES:
147         t3[c] = {i: [(p, t, t + SERVICE) for (p, t) in summary[c]['P3_times'][i]]
148                     for i in range(len(summary[c]['P3_times']))}
149     save_result_excel('result3', s3, t3, os.path.join(RES, 'result3.xlsx'), p3=True)
150
151     # ----- 图 -----
152     for c in CASES:
153         pts = load_case(c)
154         coords = [None] + [(p[1], p[2]) for p in pts]
155         zones = load_zones(c)
156         # P1 路线
157         plot_routes(c, summary[c]['P1_sol'],
158                     f'{c} 问题一 路径规划 (N={summary[c]["Nmin"]}, Tmax={summary[c]["P1
    ↪ "]["Tmax"]})h)',
159                     os.path.join(FIG, f'{c}_P1.png'), coords=coords)
160         # P3 路线 + 禁飞区
161         plot_routes(c, summary[c]['P3_sol'],
162                     f'{c} 问题三 路径规划(含禁飞区) (N={summary[c]["P3"]["N"]}, Tmax={
    ↪ summary[c]["P3"]["Tmax"]})h)',
163                     os.path.join(FIG, f'{c}_P3.png'), zones=zones, coords=coords)
164         # 负载条形图
165         fig, ax = plt.subplots(figsize=(7, 3.2))
166         x = np.arange(summary[c]['Nmin'])
167         w = 0.3
168         ax.bar(x - w, summary[c]['P1']['loads'], w, label='问题一')
169         ax.bar(x, summary[c]['P2']['loads'], w, label='问题二')

```



```

170     ax.bar(x + w, summary[c]['P3']['route_times'], w, label='问题三')
171     ax.axhline(9, color='r', ls='--', lw=1, label='9小时')
172     ax.set_xticks(x)
173     ax.set_xticklabels([f'UAV{i+1}' for i in x])
174     ax.set_ylabel('工作时间 (h)')
175     ax.set_title(f'{c} 无人机负载对比')
176     ax.legend(fontsize=8)
177     plt.tight_layout()
178     plt.savefig(os.path.join(FIG, f'{c}_loads.png'), dpi=130)
179     plt.close()
180     print(f'figs {c} done', flush=True)
181
182     # 保存 summary (去掉sol等大对象)
183     clean = {}
184     for c in CASES:
185         clean[c] = {k: v for k, v in summary[c].items() if not k.endswith('_sol') and
186                     ↪ k != 'P3_times'}
187     json.dump(clean, open(r'D:/A题_work/summary_all.json', 'w', encoding='utf-8'),
188               ensure_ascii=False, indent=1)
189     print('ALL DONE')
190     print(json.dumps(clean, ensure_ascii=False, indent=1))
191
192 if __name__ == '__main__':
193     main()

```